

Java Literals, Numeric Suffixes, and Default Integer & Decimal Behavior

Learning Objectives

After completing this topic, you will be able to:

- Understand what literals are in Java.
 - Differentiate between various types of literals.
 - Understand integer and floating-point literals.
 - Learn numeric suffixes (L, F, D).
 - Understand Java's default data types for numbers.
 - Avoid common mistakes involving literals.
 - Answer tricky interview questions confidently.
-

1. What is a Literal?

A **literal** is a **fixed value** written directly in the source code.

In simple words:

A literal is the actual value you type into your Java program.

Example:

```
int age = 22;
```

Here:

- 22 → Integer Literal
- age → Variable

Another example:

```
double pi = 3.14159;
```

3.14159 is a floating-point literal.

2. Why Do Literals Exist?

Without literals, Java would have no actual values to assign.

Example:

```
int x = 10;
```

Java needs the value 10 to initialize x.

3. Types of Literals in Java

Java supports several kinds of literals:

Literal Type	Example
Integer	100
Floating Point	3.14
Character	'A'
String	"Java"
Boolean	true, false
Null	null

Today's focus is on **numeric literals**.

4. Integer Literals

Integer literals represent **whole numbers**.

Examples:

```
10  
250  
-50  
0  
999999
```

Usage:

```
int age = 22;  
int marks = 95;
```

5. Integer Literal Formats

Java allows integer literals in four number systems.

Decimal (Base 10)

Most common.

```
int x = 100;
```

Binary (Base 2)

Starts with 0b

```
int x = 0b1010;
```

Output:

```
10
```

Octal (Base 8)

Starts with 0

```
int x = 012;
```

Output:

```
10
```

Explanation:

```
012 (Octal)  
  
1 × 8 + 2  
  
= 10 decimal
```

Hexadecimal (Base 16)

Starts with 0x

```
int x = 0xA;
```

Output

```
10
```

Another:

```
int color = 0xFF;
```

Output

```
255
```

6. Underscores in Numeric Literals (Java 7+)

For readability:

```
int population = 1_500_000;
```

Same as:

```
1500000
```

Allowed:

```
1_000  
10_00  
100_000
```

Not allowed:

```
_100  
100_  
1_.5
```

7. Default Integer Type

This is one of Java's most important rules.

Whenever Java sees:

```
10
```

Java assumes:

```
int
```

NOT

```
byte  
short  
long
```

Example:

```
var x = 10;
```

Compiler treats it as:

```
int x = 10;
```

8. Why Doesn't Java Assume byte?

Suppose:

```
5
```

Could fit in:

- byte
- short
- int
- long

Java must choose **one default type**.

Java designers chose:

int

Reasons:

- Fastest on most CPUs.
 - Native integer size.
 - Avoids unnecessary conversions.
 - Simpler language rules.
-

9. Integer Overflow

```
int x = 2147483647;
```

Maximum int.

Now:

```
x++;
```

Output:

```
-2147483648
```

This is **overflow** because Java does not automatically promote an int to a long when it exceeds its range.

10. Long Literals

Suppose:

```
long number = 5000000000;
```

Compiler Error!

Why?

Because:

```
5000000000
```

is treated as an **int literal** first, and it does not fit into an int.

Correct:

```
long number = 5000000000L;
```

or

```
long number = 50000000000l;
```

Use uppercase L because lowercase l looks like 1.

11. Numeric Suffixes

Java uses suffixes to specify the intended type.

Suffix	Type
L	long
F	float

D	double (optional)
---	-------------------

Long

```
100L
```

Float

```
3.14F
```

Double

```
3.14D
```

or simply:

```
3.14
```

because double is the default.

12. Floating-Point Literals

Numbers with decimals:

```
3.14  
0.5  
99.99
```

Default type?

Always:

```
double
```

Example:

```
double pi = 3.14;
```

Perfectly valid.

13. Why Does This Fail?

```
float price = 5.5;
```

Compiler Error.

Reason:

5.5 is a double.

Java does not automatically convert a double to a float because it may lose precision.

Correct:

```
float price = 5.5F;
```

or

```
float price = (float) 5.5;
```

Using the F suffix is the preferred approach.

14. Scientific Notation

Java supports exponent notation.

```
double x = 1e3;
```

Output:

```
1000.0
```

Another:

```
double y = 5.2e2;
```

Output

```
520.0
```

15. Character Literals

Single quotes.

```
char grade = 'A';
```

NOT

```
char grade = "A";
```

because double quotes create a String.

16. String Literals

Double quotes.

```
String name = "Siraj";
```

Creates a String object.

17. Boolean Literals

Only two:

```
true  
false
```

Example:

```
boolean passed = true;
```

18. Null Literal

Only for reference types.

```
String name = null;
```

Cannot be assigned to primitive types.

Wrong:

```
int x = null;
```

Compiler Error.

19. Default Numeric Behavior Summary

Literal	Default Type
5	int
100	int
500L	long
3.14	double
3.14F	float
3.14D	double

Remember these defaults:

- Whole numbers → int
 - Decimal numbers → double
-

20. Common Mistakes



```
float x = 5.5;
```

Needs F.



```
long x = 50000000000;
```

Needs L.



```
char c = "A";
```

Must use single quotes:

```
char c = 'A';
```



```
int x = 012;
```

Many beginners expect 12, but it is interpreted as **octal** (10 decimal).

21. Best Practices

✓ Use uppercase L.

```
100L
```

instead of

```
100l
```

✓ Use F only when you really need a float. Otherwise, prefer double because it offers higher precision and is the default floating-point type.

✓ Use underscores for readability.

```
1_000_000
```

✓ Avoid leading zeros unless you intentionally want an octal literal.

22. Real-World Examples

```
int age = 22;

long population = 8_500_000_000L;

double salary = 45000.75;

float temperature = 36.5F;

char grade = 'A';

boolean active = true;

String city = "Rawalpindi";
```

23. Tricky Interview Questions

Q1. What is a literal?

Answer:

A fixed value written directly in Java source code.

Q2. What is the default type of 100?

Answer:

int

Q3. What is the default type of 3.14?

Answer:

double

Q4. Why does this fail?

```
float x = 5.5;
```

Answer:

Because 5.5 is a double. Add the F suffix or cast it to float.

Q5. Why does this fail?

```
long x = 5000000000;
```

Answer:

The literal is treated as an int first, and it exceeds the int range. Write it as 5000000000L.

Q6. Which suffix is used for long?

Answer:

L

Q7. Which suffix is used for float?

Answer:

F

Q8. Is the D suffix mandatory for double?

Answer:

No. Decimal literals are double by default.

Q9. Why is uppercase L preferred?

Answer:

Because lowercase l can easily be mistaken for the digit 1.

Q10. Can null be assigned to an int?

Answer:

No. null is only valid for reference types.

24. Output-Based Interview Questions

Question 1

```
System.out.println(10);
```

Output

```
10
```

Type of literal: int

Question 2

```
System.out.println(10L);
```

Output

```
10
```

Type of literal: long

Question 3

```
double x = 5;  
  
System.out.println(x);
```

Output

```
5.0
```

Reason: An int literal is automatically widened to a double.

Question 4

```
float x = 5.5F;  
  
System.out.println(x);
```

Output

```
5.5
```

Question 5

```
int x = 0b1010;  
  
System.out.println(x);
```

Output

```
10
```

Question 6

```
int x = 012;  
  
System.out.println(x);
```

Output

```
10
```

Reason: 012 is an octal literal.

Question 7

```
int x = 0xA;  
  
System.out.println(x);
```

Output

```
10
```

Question 8

```
double x = 1e2;  
  
System.out.println(x);
```

Output

```
100.0
```

25. Cheat Sheet

Literal	Meaning	Default Type
100	Decimal integer	int
0b1010	Binary	int
012	Octal	int
0xFF	Hexadecimal	int
100L	Long integer	long
3.14	Decimal	double
3.14F	Decimal	float
3.14D	Decimal	double
'A'	Character	char
"Java"	String	String
true	Boolean	boolean
null	Null reference	Reference types only

Top 10 Interview Questions to Memorize

1. What is a literal in Java?
2. What is the default type of an integer literal?
3. What is the default type of a decimal literal?
4. Why does `float x = 5.5;` fail?
5. Why does `long x = 5000000000;` fail?
6. What are the L, F, and D suffixes used for?
7. What are the four formats for integer literals?
8. What is scientific notation in Java?
9. Why should you prefer 100L over 100I?
10. Why are decimal literals double by default instead of float?

Interview Tip: A very common trap is assuming Java chooses the smallest type that fits a literal. It does **not**. Java has fixed defaults: **whole-number literals are int**, and

decimal literals are double, unless you explicitly use a suffix like L or F.